

MAD UNIT - 2

Made from the IPU recommended textbooks

1. *Android Programming: The Big Nerd Ranch Guide* by Bill Phillips and Chris Stewart
2. *"iOS Programming: The Big Nerd Ranch Guide"* by Christian Keur and Aaron Hillegass
3. *"Head First Android Development: A Brain-Friendly Guide"* by Dawn Griffiths and David Griffiths
4. *"Beginning iOS Cloud and Database Development: Build Data-Driven Cloud Apps for iOS"* by Thomas P. Fitzpatrick

Of course. Here is the highly detailed and expanded version of the notes for Unit 2, with the requested page numbers for the corresponding diagrams from the provided textbooks.

Unit 2: Mobile App Development Technologies

This unit delves into the architectural and technological foundations of modern mobile applications. We will explore how apps are structured, how they communicate with servers, how they manage data, and how they interact with the physical world through location services.

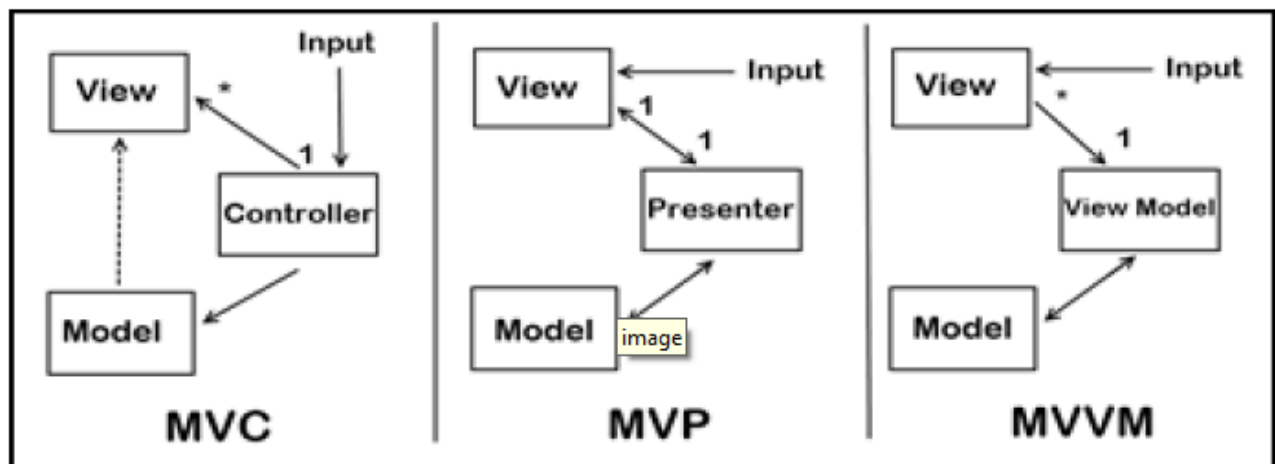
Part 1: Mobile App Architecture and Components

2.1 Mobile App Architecture

- **What is Software Architecture?**
 - **Definition:** Software architecture refers to the fundamental structures of a software system and the discipline of creating such structures. It defines how the major components of an application fit together, interact, and are managed.
 - **Importance:** A well-designed architecture is crucial for the long-term success of an application. It promotes:
 - **Maintainability:** Makes the code easier to understand, modify, and debug.
 - **Scalability:** Allows the app to grow in complexity and handle more data/users without a complete rewrite.
 - **Testability:** Isolates different parts of the code, making it easier to write unit and integration tests.
 - **Separation of Concerns:** Each component has a distinct responsibility (e.g., one part handles the UI, another handles data, another handles business logic).

- Common Architectural Patterns

Difference between MVP, MVVM, and MVC.



MVC	MVP	MVVM
The Input is directed at the controller	The Input begins with View	The input begins with model
Many-to-one relation between Controller and View	One-to-one mapping between view and its associated presenter	One-to-many mapping between various and one ViewModel
View does not have any knowledge about the controller	The View hold reference to the presenter; Presenter also reacting to the events being triggered from the View,so presenter does aware of the View	The View holds reference to the ViewModel;View Model has no information about the View.Here different technology can share viewModel (i.e) WCF and Silverlight can share same ViewModel.
Controller does pass the Model to the View,so View has knowledge about the controller	View is not Model aware	View is not Model aware

- Model-View-Controller (MVC):** A traditional pattern used extensively in software development. [FIG]
 - Model:** The data and business logic of the application. It is responsible for managing the app's state and notifying observers (the View) of changes. It has no direct knowledge of the UI.
 - View:** The user interface (UI) layer. It displays the data from the Model and sends user actions (e.g., button clicks) to the Controller.
 - Controller:** The intermediary. It receives user input from the View, processes it (by interacting with the Model), and updates the View with the results. In Android, the `Activity` or `Fragment` often acts as the Controller.
 - Challenge:** The Controller can become very large and tightly coupled to the View, leading to a "Massive Controller" problem.
- Model-View-Presenter (MVP):** An evolution of MVC that aims to improve the separation of concerns and testability. [FIG]
 - Model:** Same as in MVC (data and business logic).

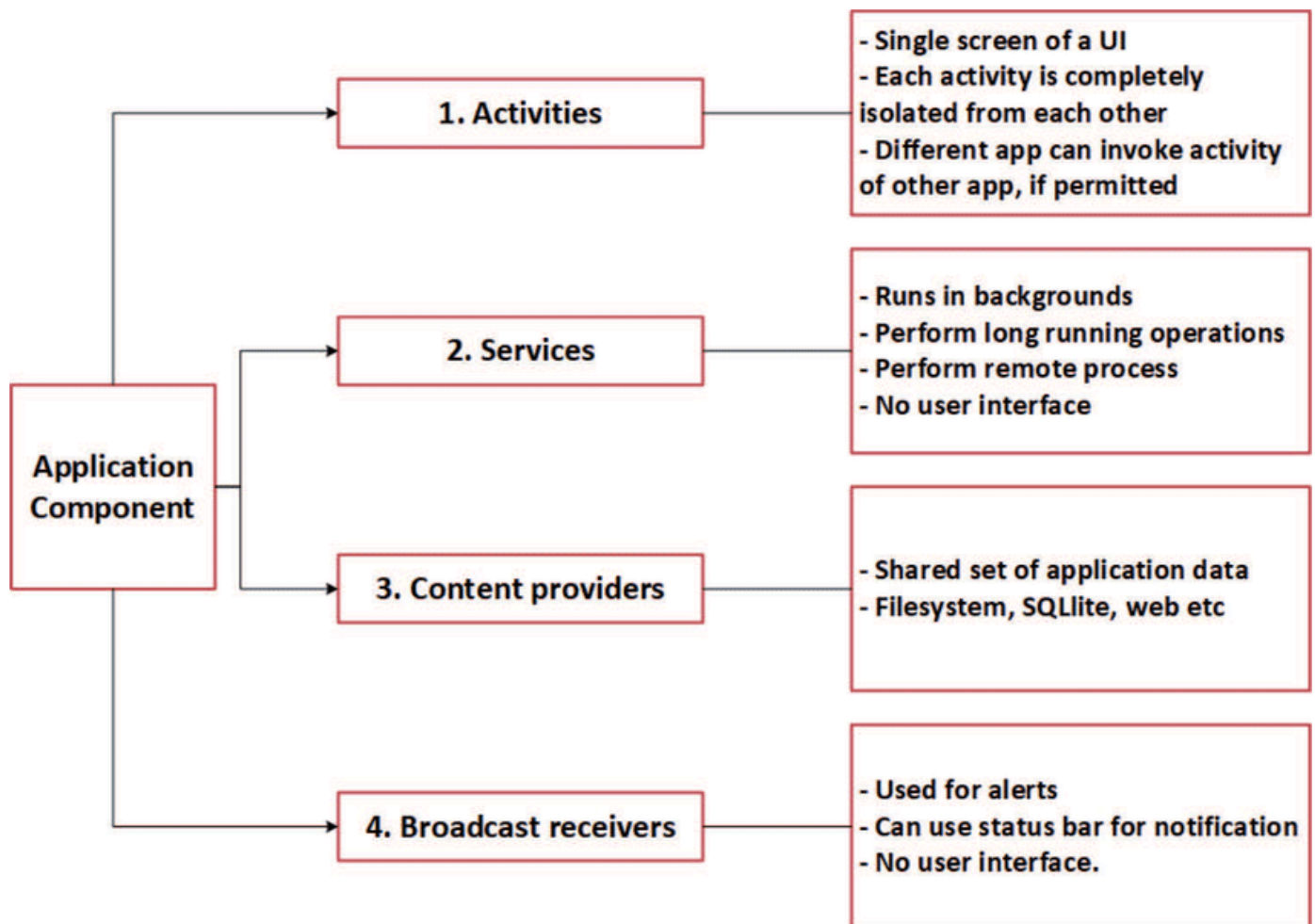
- **View:** A passive interface that displays data and routes user events to the Presenter. The View typically implements an interface, making it easily mockable for testing.
- **Presenter:** Acts as the "middle-man." It retrieves data from the Model and formats it for display in the View. It also reacts to user input from the View. Unlike the MVC Controller, the Presenter has no direct dependency on Android-specific APIs, making it a plain Java/Kotlin class that is easy to unit test.
- **Model-View-ViewModel (MVVM):** A modern pattern promoted by Google for Android development. [FIG]
 - **Model:** Same as in MVC/MVP.
 - **View:** The UI layer (Activity, Fragment). Its primary role is to observe the ViewModel for data changes and update itself accordingly. It informs the ViewModel about user actions.
 - **ViewModel:** Holds and prepares UI-related data. It exposes data streams (often using `LiveData` or `StateFlow`) that the View can observe. The ViewModel is lifecycle-aware, meaning it can survive configuration changes (like screen rotation) that destroy and recreate the View, preventing data loss.

• **Tabular Difference: Architectural Patterns**

Feature	MVC (Model-View-Controller)	MVP (Model-View-Presenter)	MVVM (Model-View-ViewModel)
View's Role	Displays data, sends user actions to Controller.	Passive; displays data given by Presenter, routes user actions.	Observes ViewModel for data changes; informs ViewModel of user actions.
Intermediary	Controller	Presenter	ViewModel
Coupling	View and Controller are tightly coupled.	View and Presenter are decoupled via an interface.	View and ViewModel are decoupled via data binding and observables.
Testability	Difficult to unit test the Controller.	Easy to unit test the Presenter.	Easy to unit test the ViewModel.
Lifecycle Handling	Poor; Controller is tied to the View's lifecycle.	Better; Presenter is a plain class but still needs manual lifecycle handling.	Excellent; ViewModel is lifecycle-aware and survives configuration changes.

2.2 Core Application Components (Android)

Android apps are not monolithic; they are constructed from several distinct types of components. Each component type has a specific purpose and lifecycle.



- **Activities**

- **Definition:** An Activity is a single, focused screen with which a user can interact. It is the primary component for user interaction.
- **Purpose:** To provide a window for the app's UI. An app is typically a collection of activities (e.g., a login activity, a list activity, a detail activity).

- **Services**

- **Definition:** A component that runs in the background to perform long-running operations without providing a user interface.
- **Purpose:** Ideal for tasks that should continue even when the user leaves the app, such as playing music, downloading a large file, or syncing data with a server.
- **Types:**
 - **Started Service:** Runs in the background indefinitely until it completes its work and stops itself.
 - **Bound Service:** Offers a client-server interface that allows other components to bind to it, send requests, and get results. It runs only as long as another component is bound to it.

- **Broadcast Receivers**

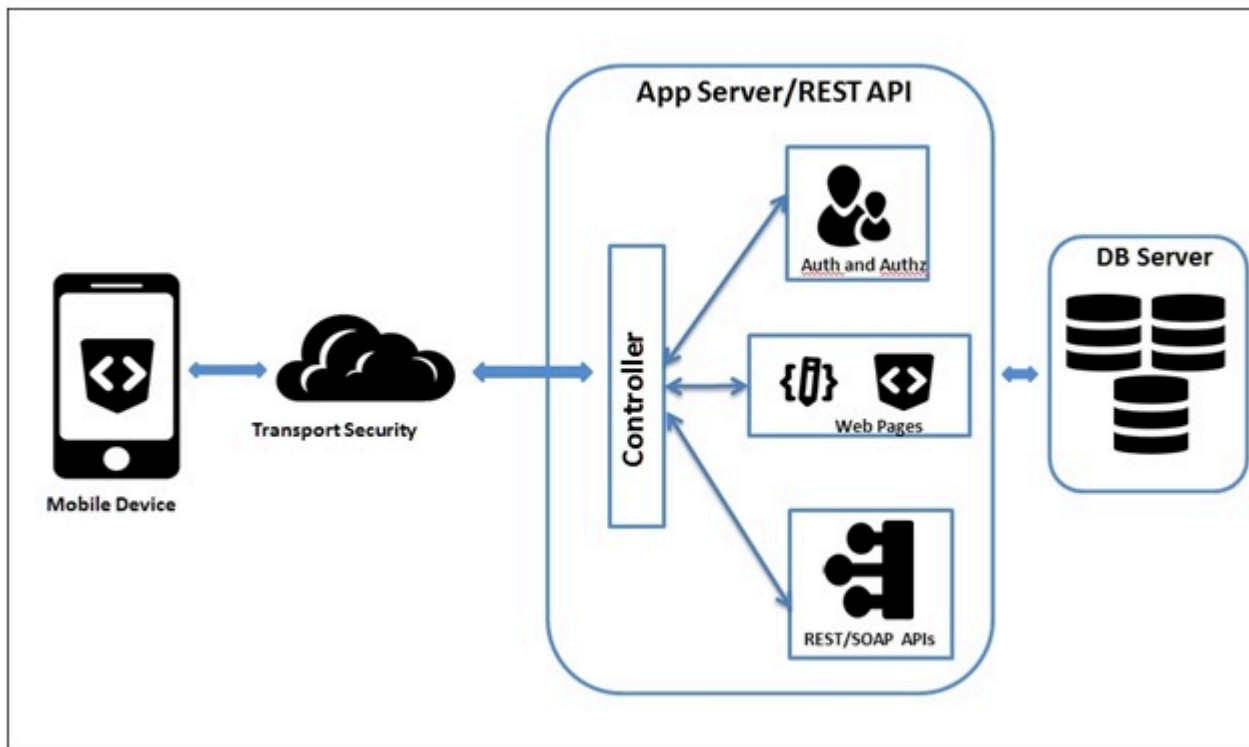
- **Definition:** A component that allows the system and other apps to deliver events to your app, even when it is not running.

- **Purpose:** To respond to system-wide announcements, such as the device finishing booting, the battery being low, or a photo being captured.

- **Content Providers**

- **Definition:** A component that manages a shared set of app data. It provides a structured interface for other apps to access and modify your app's data.
- **Purpose:** To share data securely between applications. For example, the Android system provides a Content Provider for managing the user's contacts, which other apps can query for permission.

Part 2: Backend Integration and API Consumption



2.3 Backend and APIs

- **The Role of a Backend (Server):**

- While some simple apps are self-contained, most modern apps are clients that communicate with a backend server. The backend is responsible for:
 - **Data Persistence:** Storing and retrieving data from a central database.
 - **Business Logic:** Executing complex rules and computations that shouldn't be done on the client.
 - **User Authentication and Management:** Handling user login, registration, and profiles.
 - **Push Notifications:** Sending notifications to devices.

- **API (Application Programming Interface):**

- **Definition:** An API is a set of rules, protocols, and tools for building software applications. In the context of backend integration, it is a contract that defines how the mobile app (the client) can communicate with the server.

- **Analogy:** Think of an API as a restaurant menu. The menu tells you what dishes (data/functionality) you can order and how to order them. You don't need to know how the kitchen (the server) prepares the dish; you just need to follow the menu.
- **REST (Representational State Transfer) APIs:**
 - The most common architectural style for creating APIs for web and mobile services.
 - **Key Principles:**
 - **Client-Server:** The client (mobile app) and server are separate. The client handles the UI, and the server handles the data and logic.
 - **Stateless:** Each request from a client to the server must contain all the information the server needs to understand and process the request. The server does not store any client state between requests.
 - **HTTP Methods:** REST APIs use standard HTTP methods to perform actions on resources.

HTTP Method	Action	Description
GET	Read	Retrieve a representation of a resource (e.g., get a user's profile).
POST	Create	Create a new resource (e.g., post a new message).
PUT	Update/Replace	Update an existing resource with new data.
DELETE	Delete	Remove a resource.

2.4 Data Formats (JSON)

- **JSON (JavaScript Object Notation):**
 - A lightweight, text-based, human-readable data interchange format. It has become the de facto standard for APIs because it is easy for both humans to read and machines to parse.
 - **Syntax:**
 - Data is in **name/value pairs** (e.g., `"firstName": "John"`).
 - Data is separated by commas.
 - Curly braces `{ }` hold **objects**.
 - Square brackets `[ ]` hold **arrays**.
- **Example JSON Object:**

```
{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com",
  "is_active": true,
  "courses": [
```

```

    { "course_id": "CS101", "name": "Intro to Computer Science" },
    { "course_id": "MA203", "name": "Calculus II" }
  ]
}

```

2.5 API Consumption Process

The process of interacting with a backend API from a mobile app typically follows these steps:

1. **Make an HTTP Request:** The app sends an HTTP request to a specific URL (endpoint) on the server.
2. **Receive an HTTP Response:** The server processes the request and sends back an HTTP response, which includes a status code (e.g., `200 OK`, `404 Not Found`) and a body containing data (usually in JSON format).
3. **Parse the Response:** The app parses the JSON data from the response body and converts it into native objects (e.g., Kotlin/Java data classes).
4. **Update the UI:** The app uses the newly parsed objects to update the user interface.

- **Example Code: Making a Network Request (Conceptual Android/Kotlin)**

This conceptual example uses Retrofit, a popular networking library for Android.

```

// 1. Define the data model to match the JSON
data class User(val id: Int, val name: String, val email: String)

// 2. Define the API endpoints in an interface
interface ApiService {
    @GET("users/{userId}")
    fun getUser(@Path("userId") id: Int): Call<User>
}

// 3. Make the call in your Activity or ViewModel
val retrofit = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()

val apiService = retrofit.create(ApiService::class.java)

apiService.getUser(123).enqueue(object : Callback<User> {
    override fun onResponse(call: Call<User>, response: Response<User>) {
        val user = response.body()
        // 4. Update the UI with the user data
        textView.text = user?.name
    }
}

```



```
        override fun onFailure(call: Call<User>, t: Throwable) {  
            // Handle network error  
        }  
    })
```

Part 3: Data Storage and Synchronization

2.6 On-Device Data Storage

- **Why Store Data Locally?**

- **Offline Access:** Allows the app to be functional even without an internet connection.
- **Performance:** Loading data from local storage is much faster than fetching it from a remote server.
- **Caching:** Store frequently accessed data locally to reduce network usage and improve responsiveness.

- **Types of Local Storage:**

- **Shared Preferences / UserDefaults:**

- **Purpose:** For storing small amounts of primitive data in key-value pairs. Ideal for user settings, preferences, or simple flags.
 - **Android Example (Kotlin):**

```
// Get a reference to SharedPreferences  
val sharedPref = getSharedPreferences("my_app_prefs",  
Context.MODE_PRIVATE)  
  
// Save a value  
with (sharedPref.edit()) {  
    putString("USERNAME", "JohnDoe")  
    apply()  
}  
  
// Read a value  
val username = sharedPref.getString("USERNAME", "DefaultUser")
```

- **Internal & External File Storage:**

- **Purpose:** For storing larger, unstructured files like images, audio, video, or documents.
 - **Internal Storage:** Private to your application. Other apps cannot access these files, and they are removed when the app is uninstalled.
 - **External Storage:** Files stored in a shared location on the device. Other apps can access these files (with permission).

- **SQLite Database:**

_id	NAME	DESCRIPTION	IMAGE_RESOURCE_ID
1	"Latte"	"Espresso and steamed milk"	54543543
2	"Cappuccino"	"Espresso, hot milk and steamed-milk foam"	654334453
3	"Filter"	"Our best drip coffee"	44324234

- **Purpose:** For storing large amounts of structured, private data. SQLite is a lightweight, serverless, transactional SQL database engine that is built into Android and iOS. [FIG]
- **Structure:** Data is organized into tables, rows, and columns, and you interact with it using the SQL query language.
- **Android `SQLiteOpenHelper`:** A helper class that manages database creation and version management.
- **Example Code: Creating a Table in Android**

```
// Inside a subclass of SQLiteOpenHelper
@Override
public void onCreate(SQLiteDatabase db) {
    String SQL_CREATE_TABLE = "CREATE TABLE " + TABLE_NAME + " ("
        + "_id INTEGER PRIMARY KEY AUTOINCREMENT, "
        + "name TEXT NOT NULL, "
        + "description TEXT);";
    db.execSQL(SQL_CREATE_TABLE);
}
```

2.7 Data Synchronization

- **Definition:** The process of keeping data on multiple devices and/or a central server consistent and up-to-date.
- **Synchronization Strategies:**
 - **Manual Sync:** The user initiates the sync process, typically via a "refresh" button or "pull-to-refresh" gesture.
 - **Automatic Sync:** The app automatically syncs data at periodic intervals or in response to system events (e.g., when the app launches or when network connectivity is restored).
 - **Push-based Sync:** The server initiates the sync by sending a silent push notification to the device when new data is available. This is the most efficient method for real-time updates. Services like **Firebase Cloud Messaging (FCM)** for Android and **Apple Push Notification Service (APNS)** for iOS are used for this.

Part 4: Location-Based Services and Mapping

2.8 Location Services

- **Core Concepts:**

- **Location Providers:** The sources of location data on a device.
 - **GPS (Global Positioning System):** Highly accurate (outdoor), but consumes significant battery.
 - **Network Provider:** Uses cell towers and Wi-Fi signals to determine location. Less accurate than GPS, but faster and uses less power.
 - **Fused Location Provider (Android):** A smart API that automatically chooses the best provider (or a combination) based on the app's accuracy and power requirements.
- **Permissions:** Accessing a user's location is a major privacy concern.
 - **Explicit Permission:** Your app *must* explicitly request permission from the user before accessing their location.
 - **Android Permissions:** You must declare either `ACCESS_COARSE_LOCATION` (network-based) or `ACCESS_FINE_LOCATION` (GPS-based) in your `AndroidManifest.xml` and request it from the user at runtime.

- **Getting Location Updates:**

- The general process involves:
 1. Requesting permission from the user.
 2. Getting an instance of the `LocationManager` or `FusedLocationProviderClient`.
 3. Creating a location request object to specify accuracy and update interval.
 4. Registering a listener or callback that will be triggered with new location data.
- **Example Code: Requesting Location Updates (Conceptual Android)**

```
// Assuming permissions have been granted
fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this)
locationRequest = LocationRequest.create().apply {
    interval = 10000 // 10 seconds
    fastestInterval = 5000 // 5 seconds
    priority = LocationRequest.PRIORITY_HIGH_ACCURACY
}

locationCallback = object : LocationCallback() {
    override fun onLocationResult(locationResult: LocationResult?) {
        locationResult ?: return
        for (location in locationResult.locations){
            // Update UI with the new location
        }
    }
}
```

```
fusedLocationClient.requestLocationUpdates(locationRequest,
locationCallback, Looper.getMainLooper())
```

2.9 Mapping

- **Mapping Services:** Platforms that provide map data and tools to display it.
 - **Google Maps Platform:** The standard for Android. Provides powerful APIs for maps, places, and routes.
 - **MapKit (iOS):** Apple's native mapping framework.
- **Core Mapping Components:**
 - **Map View:** The UI component that displays the map.
 - **Markers (Pins/Annotations):** Icons used to mark specific points on a map.
 - **Polylines and Polygons:** Used to draw lines, routes, or shapes on the map.
- **Example Code: Adding a Marker to a Google Map (Android)**

```
// In a class that implements OnMapReadyCallback
@Override
public void onMapReady(GoogleMap googleMap) {
    mMap = googleMap;

    // Add a marker in Sydney and move the camera
    LatLng sydney = new LatLng(-34, 151);
    mMap.addMarker(new MarkerOptions().position(sydney).title("Marker in
Sydney"));
    mMap.moveCamera(CameraUpdateFactory.newLatLng(sydney));
}
```
